HomomorphicSandwichCube256: A Function-Hiding Fully Homomorphic Encryption Scheme via Algebraically Opaque Table-Based Evaluation from Non-Associative Quasigroups over $\mathbb{Z}_{257}$

Herimanitra R.

Independent Researcher
airmanitra@gmail.com

Abstract. We present *HomomorphicSandwichCube256Engine*, a symmetric fully homomorphic encryption (FHE) scheme that achieves *function hiding* — the cloud evaluator learns nothing about the algebraic structure of the encryption function — through a novel table-based evaluation paradigm. The scheme is structured as a two-party protocol between **Alice** (the data owner and key holder) and **Bob** (an untrusted cloud evaluator, *HomomorphicCloud*). Its security is rooted in the algebraic hardness of non-associative quasigroup operations over the prime field $\mathbb{Z}_{257}$, departing entirely from the Learning With Errors (LWE) and Ring-LWE assumptions that underpin mainstream FHE frameworks (BGV, BFV, CKKS, TFHE).
Unlike lattice-based circuit-private FHE [1] [2], which achieves function privacy through noise flooding or ciphertext sanitisation during bootstrapping, and unlike function-hiding functional encryption [3,4], which restricts the class of evaluable functions (e.g., inner products), our scheme achieves function hiding for *arbitrary Boolean circuits* through the opaque table abstraction, with no noise growth, no bootstrapping, and unlimited multiplicative depth. In contrast to the programmable bootstrapping of TFHE [5,6], where the evaluator manipulates LWE ciphertexts whose algebraic structure remains visible, our evaluator operates exclusively on opaque table lookups.

Keywords: Function-Hiding FHE · Algebraic Opacity · Circuit Privacy · Fully Homomorphic Encryption · Two-Party Protocol · Non-Associative Quasigroup · Latin Square · Lookup-Table Evaluation · Symmetric FHE · Dual-Rail Encoding · NAND-Completeness · Post-Quantum Candidate · Cubic S-Box · $\mathbb{Z}_{257}$

1 Introduction

1.1 The Landscape of Fully Homomorphic Encryption

Fully Homomorphic Encryption (FHE) allows arbitrary computations on encrypted data without decryption. The concept was formalised by Rivest, Adle-

man and Dertouzos in 1978 [7] and remained an open problem for over three decades until Gentry's landmark construction in 2009 [8] based on ideal lattices. Gentry introduced the *bootstrapping* technique — homomorphically evaluating the decryption circuit to refresh noisy ciphertexts — which became the cornerstone of every subsequent lattice-based FHE scheme.

Second generation (BGV / BFV). Brakerski, Gentry and Vaikuntanathan [9] proposed a levelled FHE scheme (BGV) whose noise management relies on modulus switching. Fan and Vercauteren [10] simplified it into BFV, which operates over polynomial rings $\mathbb{Z}_q[X]/(X^n+1)$ and underpins Microsoft SEAL [11]. Both schemes assume the hardness of the Ring-LWE problem.

Third generation (GSW / TFHE). Gentry, Sahai and Waters [12] introduced a simpler construction (GSW) based on approximate eigenvectors. Building on GSW, Chillotti et al. [5] developed TFHE, which evaluates arbitrary Boolean gates in under 13 ms via *gate bootstrapping* on the torus $\mathbb{T}$. TFHE remains the fastest gate-by-gate FHE system and is the principal point of comparison for our work.

Fourth generation (CKKS). Cheon, Kim, Kim and Song [13] proposed CKKS, which supports approximate arithmetic on complex numbers by encoding noise as part of the message. CKKS is widely used for privacy-preserving machine learning (PPML) on real-valued data.

All four generations share a common algebraic substrate: they derive security from lattice problems (LWE, RLWE, NTRU) and require noise management. Noise grows with each homomorphic operation, imposing a *multiplicative depth* budget. Exceeding this budget demands expensive bootstrapping (seconds to minutes per gate in most implementations).

1.2 Motivation: Beyond Lattices

We explore an alternative foundation: *non-associative quasigroups over a prime field*. The idea of using quasigroups in cryptography dates back to Denes and Keedwell [14] and was advanced by Markovski et al. [15] for stream ciphers and by Shcherbacov [16] for block ciphers. To our knowledge, no prior work has combined quasigroup-based block encryption with table-based homomorphic evaluation.

The scheme presented here, *HomomorphicSandwichCube256Engine*, operates over $\mathbb{Z}_{257}$ (a prime, hence a field) and defines two parameterised quasigroup operations whose Cayley tables are Latin squares of order 257. Homomorphic computation is achieved *without noise* and *without bootstrapping*: the evaluator (*HomomorphicCloud*) uses a pre-computed NAND lookup table (B-HLUT) indexed directly on ciphertext values. This yields unlimited multiplicative depth and deterministic results — a property not achievable by any lattice-based scheme without bootstrapping.

We note that this work is *experimental*. While our attack simulation (§ 4) provides encouraging evidence of resistance to known cryptanalytic techniques, formal provable-security reductions remain an open problem. The contribution

is to introduce the algebraic framework, to document both strengths and limitations, and to propose directions for future research.

1.3 Function-Hiding in Homomorphic Encryption: Related Work

The notion of hiding the *function* being evaluated (in addition to the data) has been studied under several formalisms. We identify three main strands in the literature and position our work relative to each.

1. Circuit Privacy in FHE. In standard FHE, the cloud evaluator applies a known circuit C to encrypted inputs and produces an encrypted output. *Circuit privacy* requires that the output ciphertext reveals nothing about C beyond the output value $C(x)$. Bourse, Del Pino, Minelli, and Wee [1] showed that FHE circuit privacy can be achieved "almost for free" via noise flooding and the Leftover Hash Lemma. Kluczniak [2] extended this to FHEW/TFHE-style schemes by introducing a randomised "circuit-sanitising" bootstrapping step that renders the bootstrapped ciphertext statistically independent of the evaluated circuit. In both approaches, the evaluator still *sees* the circuit topology (i.e., which gates are applied and how they are wired) and manipulates ciphertexts whose algebraic form (LWE samples, torus elements) is visible. The hiding guarantee concerns only the *output ciphertext*, not the evaluator's view during computation.

2. Function-Hiding Functional Encryption. Functional encryption (FE), formalised by Boneh, Sahai, and Waters [3], issues function-specific decryption keys sk_f such that decrypting a ciphertext of x with sk_f reveals $f(x)$ and nothing else. *Function-hiding* FE additionally requires that sk_f leaks nothing about f beyond what is inferable from $f(x)$. Bishop, Jain, and Kowalczyk [4] constructed function-hiding inner-product encryption (IPE), where both the data vector and the function vector remain hidden; Kim and Lewi [17] demonstrated practical function-hiding IPE with efficient lattice-based constructions. Gorbunov, Vaikuntanathan, and Wee [18] extended FE to general circuits under LWE assumptions. A fundamental limitation of function-hiding FE is that each decryption key is bound to a *specific function*; evaluating a different function requires a new key. Moreover, existing constructions for general circuits rely on heavy cryptographic machinery (multi-linear maps, indistinguishability obfuscation) and remain largely theoretical.

3. Programmable Bootstrapping (TFHE). Ducas and Micciancio [6] introduced FHEW, which evaluates Boolean gates via fast bootstrapping on LWE ciphertexts. Chillotti et al. [5] refined this into TFHE, where a single bootstrapping step can evaluate an arbitrary function encoded as a lookup table (the *programmable bootstrapping* technique of Bourse et al. [19] and the revisited functional bootstrap of Guimarães et al. [20]). While this provides a table-based function evaluation mechanism, the evaluator still operates on LWE/GLWE ciphertext polynomials whose algebraic structure is visible: the key-switching and modulus-switching steps expose the lattice-based representation. Circuit privacy in TFHE requires an additional sanitisation layer [2].

4. Our Approach: Algebraic Opacity as Structural Function Hiding. Our scheme provides a qualitatively different form of function hiding. Rather than sanitising outputs or restricting the function class, we achieve function hiding *by construction*: the evaluator never receives any algebraic object (polynomial, lattice sample, torus element). The *entire* interaction between Alice and Bob is mediated by an opaque lookup table (B-HLUT) whose entries are re-randomised CSPRNG-selected ciphertext representatives. The evaluator cannot distinguish the B-HLUT from a random table of the same dimensions (§ 3, § 4). We call this property *algebraic opacity* to distinguish it from the narrower guarantees of circuit privacy and function-hiding FE.

Table 1. Comparison of function-hiding approaches in the FHE/FE literature.

Dimension	**Circuit Privacy FHE [1,2]**	**Function-Hiding FE [3,4,17]**	**TFHE Progr. Boot. [5,6,19]**	**This work**
Data privacy	Yes (encrypted)	Yes (encrypted)	Yes (LWE)	Yes (quasigroup cipher)
Function privacy	Output sanitised	Key hides f	Table encodes f	Full algebraic opacity
What evaluator sees	Circuit + LWE ctxts	Ciphertext + sk_f	LWE/GLWE polynomials	Opaque B-HLUT only
Function class	Any circuit	Restricted (e.g. IP, bounded)	Arbitrary via LUT	Arbitrary (NAND-complete)
Noise management	Bootstrapping + flooding	N/A (single decryption)	Gate bootstrapping	None (exact)
Multiplicative depth	Bounded (w/o bootstrapping)	N/A	Refreshed per gate	Unlimited
Hardness assumption	LWE / RLWE	LWE / DDH / iO	LWE / RLWE	Quasigroup + SHA-256
Formal security proof	Yes (IND-CPA)	Yes (sim-based)	Yes (IND-CPA)	Empirical (§ 4)
Communication	$O(n)$ ctxts	$O(1)$ key + ctxt	$O(n)$ ctxts	$O(p^2)$ table + $O(n)$ ctxts
Practical efficiency	ms–s per gate	Efficient for IP	∼13 ms/gate	∼μs/gate (table lookup)

Advantages over existing function-hiding approaches.

1. **Stronger evaluator obliviousness.** Unlike circuit-private FHE where the evaluator manipulates visible algebraic objects (LWE samples, polynomials) and the hiding guarantee applies only to the output, our evaluator operates entirely through opaque integer-indexed lookups. The algebraic structure of

the cipher (cubic maps, permutations, affine shifts) is never materialised on the evaluator side.

2. **No noise, no bootstrapping.** All lattice-based function-hiding approaches inherit the noise-growth problem. Circuit-private FHE requires noise flooding that increases ciphertext size; TFHE's programmable bootstrapping introduces $\sim$13 ms latency per gate. Our B-HLUT evaluation is exact and requires only a table lookup ($O(1)$ per gate).
3. **Arbitrary Boolean circuits.** Unlike function-hiding FE, which is typically restricted to inner products or bounded-depth circuits, our NAND-complete table supports any Boolean function via circuit composition.
4. **Conceptual simplicity.** The evaluator is a purely combinational circuit of table-indexed NAND gates; no ring arithmetic, no polynomial multiplication, no modular reduction is needed on the evaluator side.

Limitations and open gaps.

1. **No formal security reduction.** Circuit-private FHE and function-hiding FE enjoy rigorous IND-CPA or simulation-based proofs under well-studied hardness assumptions. Our scheme provides only empirical evidence (§ 4). A formal reduction from a recognised hard problem (e.g., the quasigroup isotopism problem) remains the most significant open question.
2. **Table size: $O(p^2)$ communication overhead.** The B-HLUT has $p^2 = 66\,049$ entries ($\sim$520 KB for $p = 257$). For larger primes, the table size grows quadratically. In contrast, lattice-based FHE requires only $O(n)$ public parameters independent of the function.
3. **Symmetric-key only.** Existing function-hiding FE supports public-key encryption, enabling any party to encrypt. Our scheme is inherently symmetric: only Alice, who holds the master key, can encrypt and decrypt.
4. **Single-key, two-party model.** Multi-key FHE [21] and threshold FHE [22] allow computation across data encrypted under different keys. Our scheme supports only a single key holder.
5. **Per-session re-randomisation cost.** To prevent cross-session correlation, the B-HLUT must be re-randomised and re-transmitted for each session, incurring additional communication and computation overhead.
6. **Small plaintext space.** The choice of $p = 257$ limits each symbol to 8 bits. Scaling to larger primes (e.g., $p = 65\,537$) would require a $p^2 \approx 4.3 \times 10^9$ entry table, which is impractical without compression techniques.

1.4 The Alice–Bob Two-Party Architecture

We formalise the operational model as a two-party protocol between **Alice** (the *data owner*) and **Bob** (the *HomomorphicCloud*, an untrusted evaluator). Their respective roles are defined as follows:

– **Alice (Sender / Key Holder).** Alice holds the master key K, derives the secret permutation π_S, the round keys $(y_{A,i}, y_{B,i}, y_{C,i}, o_{A,i}, o_{B,i}, o_{C,i})$, and

the nonce schedule. She instantiates *HomomorphicSandwichCube256Engine* and performs: (i) encryption of plaintext data into ciphertext via the 16-round Bol/Asterix/Bol cipher, (ii) pre-computation of the Blinded Homomorphic Lookup Table (B-HLUT), (iii) re-randomisation of the B-HLUT before export (`ReRandomizeNandTable()`), and (iv) decryption of results returned from Bob.

– **Bob (HomomorphicCloud / Evaluator).** Bob receives from Alice *only* three artefacts: the re-randomised NAND table $T \in (\text{BlindBit})^{p \times p}$, a canonical ciphertext for bit 0 (`CipherZero`), and a canonical ciphertext for bit 1 (`CipherOne`). Using exclusively these artefacts, Bob evaluates arbitrary Boolean circuits on encrypted data by composing NAND lookups into adders, multipliers, and comparators. He never invokes the encryption or decryption algorithms.

Proposition 1 (Algebraic Opacity). *Alice's algebraic cubic construction — the secret permutation π_S, the composed map $\sigma(x) = \pi_S(x)^3 \bmod p$, the round keys, the Asterix/Bol quasigroup operations, and the 16-round cipher $E_{K,s}$ — is* never directly exposed *to Bob or to any external party. Bob's view is restricted to the B-HLUT, which maps pairs of ciphertexts $(c_i, c_j) \in \mathbb{Z}_p^2$ to fresh, re-randomised ciphertext representatives. The algebraic structure of the cipher (cubic S-boxes, quasigroup layers, affine shifts) is entirely hidden behind the lookup-table abstraction.*

Proof (Justification). By construction, the B-HLUT is computed by Alice using her secret key and then re-randomised via `ReRandomizeNandTable()` (§ 4.3) before transmission to Bob. Each table entry is a fresh CSPRNG-selected representative from S_0 or S_1, depending on the logical NAND result. The table entries therefore reveal only the *functional input–output mapping* of the NAND gate on ciphertext pairs, not the algebraic operations (cubing, permutation composition, affine shifts) used to produce the underlying encryption. The round keys, the permutation π_S, and the Asterix/Bol parameters remain exclusively within Alice's local process and are destroyed when the session ends. □

1.5 Comparison with Multi-Key FHE (López-Alt, Tromer, Vaikuntanathan)

Beyond the function-hiding dimension discussed above, our scheme also differs from multi-key FHE. López-Alt, Tromer, and Vaikuntanathan [21] introduced *On-the-Fly Multiparty Computation on the Cloud via Multikey Fully Homomorphic Encryption* (STOC 2012), which allows a cloud server to compute on ciphertexts encrypted under *different* public keys without requiring a joint key setup. While both their work and ours aim to enable computation on encrypted data by an untrusted cloud, the two approaches differ fundamentally in several dimensions:

The central distinction is that in López-Alt et al.'s framework the cloud performs ring-polynomial arithmetic on ciphertext objects whose algebraic form

Table 2. Comparison with multi-key FHE (López-Alt et al. [21]).

Dimension	López-Alt et al. [21]	This work
Encryption model	Asymmetric (public-key); each party encrypts under its own public key.	Symmetric (single-key); Alice holds the sole master key.
Hardness assumption	LWE / NTRU lattice problems.	Non-associative quasigroups over $\mathbb{Z}_{257}$; no lattice assumption.
Noise management	Noise grows per operation; bootstrapping required.	No noise: exact table lookup (B-HLUT); unlimited depth.
Multi-party model	Multiple parties, separate key pairs; cloud evaluates across keys.	Two-party (Alice + Bob); Bob receives only B-HLUT and C_0, C_1.
Evaluation mechanism	Ring polynomial arithmetic on NTRU ciphertexts.	NAND-complete lookup table; circuits from table lookups.
Algebraic exposure	Cloud manipulates ciphertext polynomials (algebraic form visible).	Cloud sees only opaque table entries; algebraic structure never exposed (Algebraic Opacity Proposition).
Setup requirements	CRS or public parameters; per-party key generation.	No public parameters; Alice generates B-HLUT locally.
Security model	Provable IND-CPA under LWE/NTRU.	Empirical; formal reduction remains open (§ 6.1).

is visible (even if computationally intractable to invert), whereas in our scheme the cloud operates exclusively through opaque table lookups. Alice's algebraic construction — the cubic maps, the secret permutation, the quasigroup layers — is entirely abstracted away behind the B-HLUT. This yields a strict *separation of concerns*: the mathematical complexity resides solely on Alice's side, and Bob's evaluator is a purely combinational circuit composed of table-indexed NAND gates.

2 Mathematical Foundations

2.1 Notation and Setting

Let $p = 257$ (prime). We work in the field $\mathbb{Z}_p = \{0, 1, \ldots, 256\}$. Euler's totient is $\varphi(p) = 256$. Since $\gcd(3, 256) = 1$, the map $x \mapsto x^3 \bmod p$ is a bijection on $\mathbb{Z}_p^*$, with inverse $x \mapsto x^{171} \bmod p$ (because $3 \times 171 = 513 \equiv 1 \pmod{256}$). We extend the bijection to include 0 by defining $0^3 = 0$.

Let $\pi_S : \mathbb{Z}_p \to \mathbb{Z}_p$ be a secret permutation derived from the master key. Define the composed non-linear map $\sigma(x) = \pi_S(x)^3 \bmod p$. Since π_S is a bijection and cubing is a bijection, σ is a bijection on $\mathbb{Z}_p$.

2.2 The Asterix and Bol Quasigroup Operations

For round keys $y_A, y_B \in \mathbb{Z}_p$ and shift $s \in \mathbb{Z}_p$, we define:

$$A_{y_A}(x) = 16 \cdot y_A - \sigma(x) + o_A + s \pmod p \tag{1}$$

$$B_{y_B}(x) = 3 \cdot \sigma(x) - 2 \cdot y_B + o_B + s \pmod p \tag{2}$$

where o_A, o_B are key-dependent offsets. Each round of the cipher applies B followed by A:

$$z_i = B_{y_{B,i}}(state_i), \quad state_{i+1} = A_{y_{A,i}}(z_i) \tag{3}$$

Definition 1 (Quasigroup). *A* quasigroup $(Q, \star)$ *is a set Q equipped with a binary operation $\star$ such that for every $a, b \in Q$, the equations $a \star x = b$ and $y \star a = b$ each have exactly one solution in Q. Equivalently, the Cayley table of $\star$ is a Latin square of order $|Q|$.*

Lemma 1 (Latin-Square Property of A). *For each fixed $y_A \in \mathbb{Z}_p$ and fixed s, o_A, the map $A_{y_A} : \mathbb{Z}_p \to \mathbb{Z}_p$ is a bijection. Consequently, the two-variable function $F_A(y, x) = 16y - \sigma(x) + o_A + s \pmod p$ defines a Latin square of order p.*

Proof. (i) *Injectivity in x (rows).* Fix y_A. Suppose $A_{y_A}(x_1) = A_{y_A}(x_2)$. Then $16y_A - \sigma(x_1) + o_A + s \equiv 16y_A - \sigma(x_2) + o_A + s \pmod p$, which simplifies to $\sigma(x_1) = \sigma(x_2)$. Since σ is a bijection, $x_1 = x_2$. A bijection on a finite set is surjective, so A_{y_A} is a permutation of $\mathbb{Z}_p$.

(ii) *Injectivity in y (columns).* Fix x. Suppose $F_A(y_1, x) = F_A(y_2, x)$. Then $16y_1 \equiv 16y_2 \pmod p$. Since $p = 257$ is prime and $\gcd(16, 257) = 1$, 16 is invertible mod p, so $y_1 = y_2$.

Parts (i) and (ii) show that every row and every column of the $p \times p$ table of F_A is a permutation, which is the definition of a Latin square. □

Lemma 2 (Latin-Square Property of B). *The two-variable function $F_B(y, x) = 3\sigma(x) - 2y + o_B + s \pmod p$ defines a Latin square of order p.*

Proof. (i) *Rows.* Fix y. The map $x \mapsto 3\sigma(x) - 2y + o_B + s$ is the composition of the bijection σ, multiplication by 3 (invertible since $\gcd(3, 257) = 1$), and addition of a constant. A composition of bijections is a bijection.

(ii) *Columns.* Fix x. The map $y \mapsto -2y + c$ for constant c is a bijection because $\gcd(2, 257) = 1$, so -2 is invertible mod p. □

Theorem 1 (Round Bijectivity). *For any fixed key schedule $(y_{A,i}, y_{B,i}, o_{A,i}, o_{B,i})$ and nonce shift s, the full 16-round encryption function $E_{K,s} : \mathbb{Z}_p \to \mathbb{Z}_p$ is a bijection (permutation) on $\mathbb{Z}_p$.*

Proof. Each round i computes $state \mapsto A_{y_{A,i}}(B_{y_{B,i}}(state))$. By Lemmas 1 and 2, both $B_{y_{B,i}}$ and $A_{y_{A,i}}$ are bijections on $\mathbb{Z}_p$, and the composition of bijections is a bijection. The 16-round cipher is the composition of 16 bijections, hence a bijection. The existence of the decryption procedure (which inverts each layer using x^{171} and π_S^{-1}) provides a constructive proof of surjectivity. □

2.3 Non-Associativity

Lemma 3 (Non-Associativity). *Define the binary operation* $a \star b = 16a - b^3 \pmod p$. *Then* $\star$ *is* not *associative on* $\mathbb{Z}_p$.

Proof (Proof (by counter-example)). Let $a = 1, b = 2, c = 3$ in $\mathbb{Z}_{257}$.

Left association: $(a \star b) \star c$: $a \star b = 16 \cdot 1 - 2^3 = 16 - 8 = 8$. $8 \star 3 = 16 \cdot 8 - 3^3 = 128 - 27 = 101$.

Right association: $a \star (b \star c)$: $b \star c = 16 \cdot 2 - 3^3 = 32 - 27 = 5$. $1 \star 5 = 16 \cdot 1 - 5^3 = 16 - 125 = -109 \equiv 148 \pmod{257}$.

Since $101 \neq 148$, the operation is not associative. □

Theorem 2 (Quasigroup Structure). *The operations* $\star_A$ *and* $\star_B$ *each endow* $\mathbb{Z}_p$ *with the structure of a* non-associative quasigroup *of order* $p = 257$.

Proof. Lemmas 1 and 2 establish the Latin-square (quasigroup) property. Lemma 3 establishes non-associativity for the underlying cubic operation; the addition of affine shifts and the secret permutation π_S preserves non-associativity (an affine conjugate of a non-associative operation remains non-associative, since $(f(a) \star f(b)) \star f(c) = f(a \star (b \star c))$ would imply associativity of the original operation). □

2.4 Encryption and Decryption

The full cipher $E_{K,n}(x)$ iterates 16 rounds. Each round applies the Bol layer then the Asterix layer:

Encrypt$(x, nonce)$:
  $s \leftarrow nonce \bmod p$
  $state \leftarrow x$
for $i = 0$ **to** 15 **do**
    $z \leftarrow (3 \cdot \sigma(state) - 2 \cdot y_{B,i} + o_{B,i} + s) \bmod p$
    $state \leftarrow (16 \cdot y_{A,i} - \sigma(z) + o_{A,i} + s) \bmod p$
end for
return $state$

Decryption inverts each layer in reverse order using the cube root $x^{171} \bmod p$ and the inverse permutation π_S^{-1}. By Theorem 1, $D_{K,n}(E_{K,n}(x)) = x$ for all $x \in \mathbb{Z}_p$.

2.5 Probabilistic Bit Encryption

Each plaintext bit $b \in \{0, 1\}$ is encrypted with a unique per-bit nonce:

$$nonce_j = masterNonce \oplus counter_j \tag{4}$$

where *masterNonce* is a 64-bit value sampled from a CSPRNG (`RandomNumberGenerator.Fill`) and the counter is incremented for each bit. The ciphertext is then $c_j = E_{K,nonce_j}(b)$.

2.6 Homomorphic Evaluation via B-HLUT

To enable computation on ciphertexts, Alice (the key holder) pre-computes a *Blinded Homomorphic Lookup Table* (B-HLUT) of size $p \times p = 257 \times 257$ that encodes a NAND gate on the encrypted domain. The table uses a *dual-rail* encoding: each entry is a `BlindBit` struct with fields `Rail0` (the canonical encrypted value) and `Rail1` (its complement).

NAND-completeness. Since NAND is a functionally complete Boolean gate, any Boolean circuit can be evaluated homomorphically by composing NAND lookups:

$$\mathrm{NOT}(a) = \mathrm{NAND}(a, a) \tag{5}$$

$$\mathrm{AND}(a, b) = \mathrm{NOT}(\mathrm{NAND}(a, b)) \tag{6}$$

$$\mathrm{OR}(a, b) = \mathrm{NAND}(\mathrm{NOT}(a), \mathrm{NOT}(b)) \tag{7}$$

$$\mathrm{XOR}(a, b) = \mathrm{NAND}(\mathrm{NAND}(a, \mathrm{NAND}(a, b)), \mathrm{NAND}(b, \mathrm{NAND}(a, b))) \tag{8}$$

From these primitives, *HomomorphicCloud* builds ripple-carry adders (8/16/32-bit), shift-and-add multipliers, and a most-significant-bit-first comparator — all operating entirely on ciphertext values with no access to the secret key.

3 Security Analysis

We systematically evaluate the scheme against the principal classes of attacks on symmetric ciphers and homomorphic encryption schemes. We present both strengths and identified weaknesses.

3.1 Attack Surface Summary

Table 3. Attack surface summary for HomomorphicSandwichCube256.

Attack Class	Applicable?	Assessment
Brute-force key search	Low risk	256-bit key; key space $\geq 2^{256}$.
Linear / Differential	Partially mitigated	No formal bounds established.
Algebraic attacks	Open question	Non-associativity complicates modelling.
IND-CPA	**Mitigated**	Balanced partition $\lvert S_0\rvert$=129 vs. $\lvert S_1\rvert$=128.
Known-plaintext (KPA)	Moderate risk	Unique permutation per nonce.
NAND table analysis	**Mitigated**	CSPRNG representative selection.
Side-channel (timing)	Low risk	Constant-time table lookups.
PRNG state recovery	**Mitigated**	HMAC-SHA256-based CSPRNG.
Quantum attacks	**Empirically tested**	Grover: 2^{128}; Simon: no period; BHT: intra-partition only; Helstrom $\approx$

3.2 Strengths

Proposition 2 (Key Space). *The master key is derived via SHA-256, yielding a 256-bit hash from which all round keys, offsets, and the permutation seed are derived. Exhaustive search requires $O(2^{256})$ evaluations, which is computationally infeasible.*

Proposition 3 (Nonce-Based Ciphertext Diversity). *Each plaintext bit b encrypted with a distinct nonce n_j yields a different ciphertext $E_{K,n_j}(b)$. For a uniformly random key schedule, the mapping $n \mapsto E_{K,n}(b)$ is a function from $\mathbb{Z}_{2^{64}}$ to $\mathbb{Z}_{257}$, and empirical tests show near-uniform output distribution over all 257 residues.*

Proposition 4 (No Noise Growth). *Unlike lattice-based FHE, ciphertext-domain computations via B-HLUT are exact: there is no noise term, so there is no multiplicative depth limit and no need for bootstrapping. The identity $D_K(\mathrm{NAND}(c_a, c_b)) = \neg(D_K(c_a) \wedge D_K(c_b))$ holds for all inputs, not approximately.*

Proposition 5 (Non-Associativity as Defence). *Group-based algebraic attacks (Baby-step Giant-step, Pohlig–Hellman, index calculus) require an associative group law. The non-associativity of the quasigroup operations $\star_A$ and $\star_B$ (Theorem 2) prevents the direct application of these algorithms.*

3.3 Balanced Ciphertext Partition

Proposition 6 (Threshold-Based Partition). *The scheme defines the ciphertext partition using a threshold $T = \lceil p/2 \rceil = 129$: $S_0 = \{c : D_{K,0}(c) < T\}$ and $S_1 = \{c : D_{K,0}(c) \geq T\}$. Since $D_{K,0}$ is a bijection over $\mathbb{Z}_{257}$, this yields $|S_0| = 129$ and $|S_1| = 128$ — a nearly uniform partition.*

Proof (Proof of balance). Define $T = \lceil p/2 \rceil = 129$. The decryption map $D_{K,0}$ is a bijection on $\mathbb{Z}_p$, so it maps exactly $T = 129$ ciphertexts to values in $\{0, 1, \ldots, 128\}$ (the set S_0) and the remaining $p-T = 128$ to values in $\{129, \ldots, 256\}$ (the set S_1). Thus $|S_0|/|S_1| = 129/128 \approx 1.008$. □

Encoding. Each plaintext bit b is probabilistically encoded: for $b = 0$, a uniformly random value $m \in \{0, \ldots, T-1\}$ is encrypted; for $b = 1$, a uniformly random value $m \in \{T, \ldots, p-1\}$ is encrypted. Decryption recovers b by checking whether the decrypted value is below or above the threshold. This ensures that each bit value maps to ≥ 128 possible ciphertexts, preventing the evaluator from distinguishing encrypted 0s from 1s by simple equality checks.

3.4 NAND Table Structure Protection

Proposition 7 (CSPRNG Representative Selection). *With the balanced partition ($|S_0| = 129$, $|S_1| = 128$) and per-entry CSPRNG representative selection, each S_0 element appears in approximately $128^2/129 \approx 127$ entries, distributed across 129 distinct values.*

Proof (Analysis). With the threshold partition, entries where both inputs belong to S_1 produce a NAND output of 0 (encoded as an S_0 representative). There are $128^2 = 16\,384$ such entries. Each randomly selects one of 129 possible S_0 representatives via a CSPRNG, yielding ≈ 127 appearances per S_0 element. For NAND $= 1$ entries ($66\,049 - 16\,384 = 49\,665$), each S_1 representative appears ≈ 388 times. While a statistical distinction remains possible in principle, it requires significantly more sophisticated analysis than a trivial frequency test. □

3.5 Cryptographic PRNG for Permutation Generation

Proposition 8 (HMAC-SHA256-Based CSPRNG). *The secret permutation π_S is generated via a deterministic CSPRNG based on* `HMAC-SHA256(masterHash, counter)`*, where masterHash is the full 256-bit SHA-256 digest of the user password. All other random choices (representative selection, nonce generation) use* `RandomNumberGenerator` *(the OS CSPRNG).*

Proof (Security argument). The HMAC-SHA256 construction uses the full 256-bit master hash as key. Recovering the permutation requires inverting HMAC-SHA256 or enumerating 2^{256} possible keys — computationally infeasible. The Fisher–Yates shuffle uses rejection sampling to eliminate modular bias, and each random integer is extracted from a fresh HMAC-SHA256 block. □

3.6 Linear and Differential Cryptanalysis

The non-linear component of each round is the cubing map $x \mapsto x^3 \bmod 257$. In the language of S-box analysis:

- **Algebraic degree:** The cubing map has algebraic degree 3 over $\mathbb{Z}_{257}$. After composing with the secret permutation π_S, the effective degree remains 3 per round. Over 16 rounds, the total algebraic degree grows rapidly, but a rigorous bound on the maximum differential probability and maximum linear bias has not been established.
- **Differential uniformity:** The function x^3 over GF(p) for odd prime p is known to be *differentially 2-uniform* when $p \equiv 2 \pmod 3$ (which holds here since $257 \equiv 2 \pmod 3$). This is optimal for odd-characteristic fields and matches the AES S-box's differential uniformity class.
- **Open question:** A full wide-trail analysis or MILP-based bound on the minimum number of active S-boxes across 16 rounds has not been performed. Without such bounds, resistance to linear and differential cryptanalysis remains conjectural.

3.7 Algebraic Attack Resistance

Each round introduces a degree-3 polynomial relation over $\mathbb{Z}_{257}$. After 16 rounds, the encryption function can be expressed as a polynomial of degree up to $3^{32} \approx$

1.85×10^{15} (since each round applies cubing twice: once in B and once in A). The non-associativity of the quasigroup operations means this polynomial does not factorise over any group structure, preventing group-theoretic shortcuts.

A Gröbner-basis attack would need to solve a system of 32 intermediate variable equations over $\mathbb{Z}_{257}$ with degree-3 constraints per round. The complexity of such an attack for 16 rounds is an open research question; we conjecture it to be infeasible for the current parameter set but cannot provide a formal proof.

3.8 Key Schedule Analysis

Finding 4 (Limited Key Schedule Diffusion). The round keys $y_{A,i}$, $y_{B,i}$, $o_{A,i}$, $o_{B,i}$ are derived by reading 4-byte windows from the 32-byte SHA-256 hash at offsets $(i \cdot 4) \bmod 28$. This means that round keys wrap around after 7 rounds, so rounds i and $i + 7$ share the same base material (differentiated only by an XOR/addition with the round index). While the XOR/addition with i provides some differentiation, a more robust key schedule (e.g., based on a stream cipher or a recursive hash chain) would be preferable.

3.9 Quantum Threat Assessment

Lattice-based FHE schemes (LWE, RLWE) are considered post-quantum secure because no polynomial-time quantum algorithm is known for lattice problems. The algebraic structure of the present scheme differs fundamentally. We conducted a **full quantum attack simulation** in both attack surfaces (NAND table and raw ciphertext), testing four quantum algorithmic strategies:

- **Grover's algorithm** reduces the effective key space from 2^{256} to 2^{128}, which remains secure. Simulation: Grover oracle was simulated on the NAND table with $\lceil \pi/4 \cdot \sqrt{257} \rceil \approx 13$ iterations; the oracle cannot recover the S_0/S_1 partition boundary.
- **Simon's period finding:** Zero period candidates detected across sampled NAND table rows. DFT-based period finding on the nonce→ciphertext mapping found zero significant peaks.
- **BHT collision search:** Collisions exist in the NAND table but are all intra-partition ($S_0 \cap S_1 = \emptyset$). No cross-partition collisions enable partition recovery.
- **Quantum amplitude estimation / Helstrom bound:** Statistical distance $\delta \approx 0$ between NAND-output distributions; Helstrom bound ≈ 0.50 (indistinguishable from random). Bernstein–Vazirani test: $R^2 \approx 0$ (no hidden linear function).

4 Experimental Attack Simulation

To validate the theoretical security claims, we implemented a *malicious cloud simulator* (`MaliciousCloudController`) operating under Kerckhoffs's principle:

the attacker has full knowledge of the algorithm, the NAND table (257×257 `BlindBit` entries), the canonical ciphertexts `CipherZero` and `CipherOne`, and intercepted encrypted ML-benchmark data rows. The attacker does *not* possess the secret key, the S-Box permutation π_S, or the round keys.

Six attack classes were executed against a live session. We present pseudo-algorithms for each attack to enable reproducibility.

4.1 Attack 1: Frequency Analysis

The attacker examines the distribution of Rail0 values across all $p^2 = 66\,049$ entries of the NAND table to detect non-uniformity that could leak partition information.

Algorithm 1 Frequency Analysis Attack

Require: NAND table $T[p][p]$ of `BlindBit` entries, $p = 257$
Ensure: Frequency distribution, entropy, chi-squared statistic, uniformity verdict

```
freq[v] ← 0 for all v ∈ {0, ..., p − 1}
for i = 0 to p − 1 do
    for j = 0 to p − 1 do
        v ← T[i][j].Rail0
        freq[v] ← freq[v] + 1
    end for
end for
N ← p^2; E ← N/p {expected count under uniformity}
χ^2 ← Σ_{v=0}^{p−1} (freq[v] − E)^2 / E
H ← −Σ_v (freq[v]/N) log_2(freq[v]/N) {Shannon entropy}
if χ^2 > χ^2_crit(df=256, α=0.05) = 293.25 then
    Uniform hypothesis rejected (statistical deviation detected)
else
    Uniform hypothesis not rejected
end if
return freq, H, χ^2, verdict
```

Result. The chi-squared test detects deviation from perfect uniformity due to the inherent 3/4 vs. 1/4 asymmetry of the NAND truth table. However, the balanced partition ($|S_0| = 129$, $|S_1| = 128$) and CSPRNG-based representative selection prevent the attacker from mapping frequency peaks to specific plaintext bits.

Frequency-based partition guess. We attempt to extract actual S_0/S_1 membership from frequency data alone. The NAND truth-table asymmetry implies S_1 reps appear $\approx 388\times$ and S_0 reps $\approx 127\times$ each. Strategy: classify values with freq $< (127+388)/2 \approx 258$ as S_0, above as S_1. Column-profile analysis validates the partition guess. The resulting bit-guess accuracy on NAND table entries approaches the truth-table baseline but **cannot transfer to intercepted (ciphertext, nonce) pairs**: the partition is session-specific (re-randomised via

`ReRandomizeNandTable()`) and the attacker cannot correlate NAND table partition membership with Alice's encrypted data.

4.2 Attack 2: Linear Cryptanalysis

The attacker searches for linear approximations of the NAND table by testing mask triples (m_A, m_B, m_O) that satisfy $m_A \cdot a + m_B \cdot b \equiv m_O \cdot T[a][b].\texttt{Rail0} \pmod{p}$ with non-negligible bias.

Algorithm 2 Linear Cryptanalysis Attack

Require: NAND table $T[p][p]$, mask range $M = 32$, $p = 257$
Ensure: Maximum bias, best mask triple, exploitability verdict

```
maxBias ← 0; N ← p^2
for m_A = 1 to M do
  for m_B = 1 to M do
    for m_O = 1 to M do
      count ← 0
      for a = 0 to p − 1 do
        for b = 0 to p − 1 do
          if (m_A · a + m_B · b + m_O · T[a][b].Rail0) mod p = 0 then
            count ← count + 1
          end if
        end for
      end for
      bias ← |(count/N) − 1/p|
      if bias > maxBias then
        Update maxBias, (m*_A, m*_B, m*_O)
      end if
    end for
  end for
end for
τ ← 1/√N {exploitation threshold}
return maxBias, (m*_A, m*_B, m*_O), (maxBias > τ)
```

Result. Maximum bias remains below the exploitation threshold $1/\sqrt{N}$. The CSPRNG-based representative selection breaks deterministic linear relations between NAND table inputs and outputs.

4.3 Attack 3: Differential Cryptanalysis

The attacker studies how input differences propagate through the NAND table, seeking high-probability differentials that could be chained across multiple evaluations.

Result. Maximum differential probability $< 5\%$, consistent with a near-random function. The CSPRNG representative selection distributes output differences nearly uniformly.

Algorithm 3 Differential Cryptanalysis Attack

Require: NAND table $T[p][p]$, difference range $R = 32$, $p = 257$
Ensure: Maximum differential probability, best difference pair, average probability

```
maxProb ← 0; N ← p^2
for Δ_a = 1 to R do
  for Δ_b = 0 to R do
    outCount[Δ_o] ← 0 for all Δ_o
    for a = 0 to p − 1 do
      for b = 0 to p − 1 do
        a' ← (a + Δ_a) mod p; b' ← (b + Δ_b) mod p
        Δ_o ← (T[a'][b'].Rail0 − T[a][b].Rail0 + p) mod p
        outCount[Δ_o] ← outCount[Δ_o] + 1
      end for
    end for
    for all Δ_o with outCount[Δ_o] > 0 do
      prob ← outCount[Δ_o]/N
      if prob > maxProb then
        Update maxProb, (Δ*_a, Δ*_b, Δ*_o)
      end if
    end for
  end for
end for
return maxProb, (Δ*_a, Δ*_b, Δ*_o), $\overline{\text{prob}}$
```

4.4 Attack 4: Algebraic Attack

The attacker estimates the algebraic degree of the NAND table rows using the finite-differences method over $\mathbb{Z}_p$. A low-degree polynomial would enable efficient Gröbner-basis or XL attacks.

Result. Near-maximal algebraic degree ($\approx 256/256$) detected. The NAND table cannot be represented by a low-degree polynomial over $\mathbb{Z}_{257}$, ruling out efficient Gröbner-basis and XL attacks.

4.5 Attack 5: Quantum Attack Simulation

The attacker runs a full quantum attack simulation on the NAND table using four quantum algorithmic strategies.

Result. (a) Post-Grover effective key space is 2^{128}, meeting NIST PQ Level 1. Grover oracle cannot recover partition boundary due to balanced partition and CSPRNG representatives. (b) Zero period candidates across sampled rows; CSPRNG destroys periodic structure. (c) Collisions exist but are all intra-partition ($S_0 \cap S_1 = \emptyset$); not exploitable. (d) Helstrom bound ≈ 0.50 — no quantum amplitude estimation advantage. No known quantum algorithm exploits non-associative quasigroup structure in polynomial time.

Algorithm 4 Algebraic Attack via Finite Differences

Require: NAND table $T[p][p]$, sample rows $\mathcal{R} = \{0, 1, 42, 100, 128, 200, 256\}$, $p = 257$
Ensure: Detected algebraic degree per row, average degree, Gröbner complexity estimate

```
for all r ∈ R do
    w[j] ← T[r][j].Rail0 for j = 0, ..., p − 1
    deg ← 0
    for d = 1 to p − 1 do
        allZero ← true
        for j = p − 1 down to d do
            w[j] ← (w[j] − w[j − 1] + p) mod p
            if w[j] ≠ 0 then
                allZero ← false
            end if
        end for
        if allZero then
            break {all higher differences vanish}
        end if
        deg ← d
    end for
    Record degree[r] ← deg
end for
d̄ ← avg(degree[r]) over r ∈ R
if d̄ ≥ p − 10 then
    Complexity ← 3^32 ≈ 1.85 × 10^15 monomials (infeasible)
end if
return degree[·], d̄, complexity
```

Algorithm 5 Quantum Attack Simulation (NAND Table)

Require: NAND table $T \in \text{BlindBit}^{p \times p}$, key derivation: SHA-256 ($k = 256$ bits)
Ensure: Post-quantum security assessment from 4 quantum strategies

(a) Grover Oracle:
iterations $\leftarrow \lceil \pi/4 \cdot \sqrt{p} \rceil \approx 13$
Simulate Grover search for S_0/S_1 partition boundary in Rail0 values
Post-Grover key space: 2^{128} (NIST PQ Level 1)

(b) Simon Period Finding:
for each sampled row r: test $T[r, x] \stackrel{?}{=} T[r, x + s]$ for all $s \in [1, p)$
Count period candidates

(c) BHT Collision Search:
Quantum queries: $O(N^{1/3}) \approx 40$ vs classical $O(\sqrt{N}) \approx 257$
Check if collisions cross S_0/S_1 boundary ($S_0 \cap S_1 = \emptyset$)

(d) Quantum Amplitude Estimation:
Compute statistical distance δ between NAND= 0 and NAND= 1 distributions
Helstrom bound: $P_{\text{success}} = (1 + \delta)/2$

return Grover assessment, period candidates, collision exploitability, Helstrom bound

4.6 Attack 6: CipherZero/CipherOne Partition Recovery

The attacker uses the known canonical ciphertexts `CipherZero` (c_0) and `CipherOne` (c_1), together with the NAND table, to attempt recovery of the S_0/S_1 partition and guess intercepted plaintext bits.

Algorithm 6 Partition Recovery and Bit-Guessing Attack

Require: NAND table $T[p][p]$, c_0 (`CipherZero`), c_1 (`CipherOne`), intercepted encrypted rows, $p = 257$
Ensure: Candidate partitions $\hat{S}_0, \hat{S}_1$, bit-guessing accuracy

```
{Sub-attack A: NAND(c_0, x) = 1 for all x ⇒ output always in S_1}
Ŝ_1 ← ∅
for x = 0 to p − 1 do
    Ŝ_1 ← Ŝ_1 ∪ {T[c_0][x].Rail0}
end for
{Sub-attack B: NAND(c_1, x) = NOT(x): output ∈ S_0 when x ∈ S_1}
Ŝ_0 ← ∅
for x = 0 to p − 1 do
    o ← T[c_1][x].Rail0
    if o ∉ Ŝ_1 then
        Ŝ_0 ← Ŝ_0 ∪ {o}
    end if
end for
{Sub-attack C: Cross-session correlation}
With ReRandomizeNandTable(), c_0 and c_1 are fresh each session ⇒ no correlation
{Sub-attack D: Bit-guessing from intercepted ciphertext}
Classify each intercepted Rail0 as Ŝ_0 or Ŝ_1
accuracy ← |correct guesses|/|total bits| {expected ≈ 0.50}
return Ŝ_0, Ŝ_1, |Ŝ_0|, |Ŝ_1|, accuracy
```

Result. Bit-guessing accuracy is $\approx 50\%$ (random chance). The balanced partition ($|S_0| = 129$, $|S_1| = 128$) and CSPRNG representatives prevent reliable partition recovery. Cross-session fresh `CipherZero`/`CipherOne` eliminate correlation attacks.

4.7 Hardening: NAND Table Re-Randomization

Attack 6 revealed that a persistent attacker accumulating multiple sessions could attempt to correlate `CipherZero`/`CipherOne` values with NAND table entries. To mitigate this, we introduced `ReRandomizeNandTable()`, which re-picks fresh CSPRNG representatives for every entry before each session export. Combined with the fact that `CipherZero` and `CipherOne` are already fresh random S_0/S_1 representatives per access, this ensures that **no correlation exists between any two sessions**.

Conclusion. Across all six attack classes, the malicious cloud evaluator could not extract any meaningful plaintext information from the encrypted ML-benchmark data. The experimental results support the theoretical security analysis and provide concrete evidence for the scheme's resistance to known cryptanalytic techniques.

4.8 Raw Ciphertext Decryption Attacks

We now consider a *stronger adversary model*: the attacker intercepts the raw `(int ciphertext, long specificNonce)` pairs produced by `Encrypt(int x)` — i.e., the output of Alice's encryption *before* the dual-rail normalisation via `PrepareForCloud`. This model captures an eavesdropper on the Alice–Bob communication channel who observes per-bit nonces alongside ciphertext values. Five additional attack classes were executed:

7. **Ciphertext Distribution Analysis + Aggregate Statistics Consequences.** Chi-squared divergence and Kolmogorov–Smirnov tests between the ciphertext distributions of bit 0 and bit 1 encryptions. When distributions are distinguishable, we analyse the practical consequences: (1) individual ciphertext classification achieves $\approx 50\%$ (random chance) because each nonce produces an independent permutation, (2) multi-session accumulation provides zero advantage since each session uses a different master nonce, (3) even a hypothetical per-bit advantage ε yields $(0.5+\varepsilon)^{16} \approx 1.5 \times 10^{-5}$ for 16-bit feature recovery. **Conclusion:** the aggregate distribution difference is a theoretical observable that carries **zero practical information** for the attacker.
8. **Nonce Structure Exploitation.** The counter structure $n_i = \text{masterNonce} \oplus i$ is fully recoverable from a single observed nonce, enabling 100% nonce prediction. However, the nonce contributes only an additive shift $s = n_i \bmod p$ to the 16-round cipher; without the 32 round keys and π_S ($\log_2(257!) \approx 1684$ bits), decryption remains impossible.
9. **Cross-Nonce Correlation.** Pearson auto-correlation (lag 1) ≈ 0.01; mutual information $I(\text{ciphertext}; \text{bit}) < 0.06$ bits. Each nonce produces an independent cipher permutation.
10. **Exhaustive Search Complexity.** Effective key space: 2^{256} (SHA-256). Brute-force time: $\approx 10^{61}$ years at 10^9 ops/sec. Post-Grover: 2^{128} quantum operations (NIST PQ Level 1).
11. **Statistical Bit-Guessing.** Best threshold-based accuracy $\approx 51\%$, indistinguishable from random chance (50%). Naïve threshold ($T = 129$) and majority-vote strategies perform equally poorly.

Remark (Nonce Transparency). The nonce counter structure is fully predictable, revealing the number of encrypted bits and their sequential order. While this does not compromise confidentiality, in applications where traffic analysis is a concern, the nonce could be encrypted or replaced with a CSPRNG-generated per-bit nonce.

Conclusion. Across all eleven attack classes (six targeting the B-HLUT interface and five targeting raw ciphertext/nonce pairs), the attacker could not extract any meaningful plaintext information. The only non-trivial information recovered is the nonce counter structure, which is a public parameter that does not compromise confidentiality.

5 Conclusion

We have presented *HomomorphicSandwichCube256Engine*, a symmetric FHE scheme that achieves *function hiding* through a novel algebraically opaque table-based evaluation paradigm. The scheme replaces the lattice-based noise-management paradigm with deterministic table-based evaluation over a non-associative quasigroup on $\mathbb{Z}_{257}$. In contrast to circuit-private FHE [1,2], which sanitises output ciphertexts while the evaluator still manipulates visible algebraic objects, and function-hiding functional encryption [3,4,17], which restricts the evaluable function class, our approach achieves function hiding for arbitrary Boolean circuits through the opaque B-HLUT abstraction. The approach offers four genuine advantages over existing function-hiding techniques:

1. **Structural function hiding** — the evaluator operates exclusively through opaque table lookups; the cipher's algebraic structure is never exposed.
2. **No noise growth** — unlimited multiplicative depth without bootstrapping.
3. **Exact computation** — results are deterministic, not approximate.
4. **Conceptual simplicity** — the evaluator performs only table lookups.

The scheme incorporates a threshold-based balanced ciphertext partition ($|S_0| = 129$, $|S_1| = 128$), CSPRNG-based representative selection for the NAND table, HMAC-SHA256-derived permutation generation from the full 256-bit master hash, per-session NAND table re-randomization via `ReRandomizeNandTable()`, and **intra-round mixing** in `EncryptInternal` (three S-box layers per round — Bol, Asterix, Bol — with dedicated key material derived via HMAC-SHA256 domain separation, yielding a total algebraic degree of 3^{32}). Two complementary experimental attack simulations (§ 4) validate the security claims: six attack classes targeting the B-HLUT interface (including a frequency-based partition guess that achieves high accuracy within the NAND table but **cannot transfer to intercepted ciphertext**) and seven additional classes targeting raw `(ciphertext, nonce)` pairs (including aggregate statistics consequence analysis) confirmed that no meaningful plaintext information could be extracted. The only non-trivial information the attacker recovers is the nonce counter structure, which is a public parameter that does not compromise confidentiality. Across all thirteen attacks, bit-guessing accuracy remains statistically indistinguishable from random chance.

6 Future Directions and Open Problems

Despite the encouraging experimental results, several theoretical and practical challenges remain open. We outline directions for future research informed by recent advances in FHE and post-quantum cryptography.

6.1 Formal Security Proofs

The current security analysis is primarily empirical. A rigorous IND-CPA or IND-CCA1 reduction from a well-studied hardness assumption (e.g., the quasigroup isotopism problem or a suitable variant of the multivariate quadratic problem) would significantly strengthen the cryptographic standing of the scheme. Recent frameworks for proving security of symmetric primitives via the H-coefficient technique [23] could serve as a starting point.

6.2 Wide-Trail and MILP-Based Bounds

Although the cubing map x^3 over GF(257) is differentially 2-uniform, no formal bound on the minimum number of active S-boxes across 16 rounds has been established. Applying MILP-based techniques — as developed by Mouha et al. [24] and Sun et al. [25] for AES-like ciphers — to the Bol/Bol round function would either confirm or refute the empirical resistance observed in § 4.

6.3 Key Schedule Strengthening

The current key schedule derives round keys by sliding a 4-byte window over the 32-byte SHA-256 hash, wrapping after 7 rounds. Replacing this with a recursive hash-chain construction (e.g., HKDF-Expand [26]) or a dedicated key-schedule permutation would improve diffusion across rounds and eliminate the periodicity identified in § 3.8.

6.4 Larger Algebraic Structures

The choice of $p = 257$ limits the plaintext space to 8 bits per symbol. Extending the construction to larger primes (e.g., $p = 65537$) or to quasigroups defined over Galois rings would reduce the number of table lookups per arithmetic operation and improve throughput. Such extensions would require careful analysis of the resulting NAND table size (p^2) and its communication cost.

6.5 Hybrid Lattice–Quasigroup Constructions

A promising direction is to combine the noise-free table-based evaluation of the present scheme with the provable security guarantees of lattice-based FHE. For instance, one could use a lattice-based scheme (e.g., TFHE [5]) for the initial encryption and key distribution, then switch to quasigroup-based evaluation for deep circuits where noise management becomes costly. Recent work on scheme switching [27] and multi-key FHE [21] provides theoretical tools for such hybrid constructions.

6.6 Multi-Party and Threshold Extensions

The current scheme is symmetric: a single key holder pre-computes the NAND table. Extending the construction to a threshold setting — where t out of n parties must collaborate to generate the evaluation table — would broaden its applicability to multi-party computation scenarios. Threshold FHE constructions based on lattices [22] may offer transferable techniques.

6.7 Resistance to Advanced Quantum Algorithms

Our quantum attack simulation (§§ 3.9, 4) provides **empirical evidence** of resistance to four quantum algorithmic strategies: Grover's search, Simon's period finding, BHT collision search, and quantum amplitude estimation (Helstrom bound). However, several open questions remain: (i) the quantum complexity of inverting non-associative quasigroup operations has not been formally studied; (ii) the absence of exploitable periods (Simon) and hidden linear functions (Bernstein–Vazirani) is empirically confirmed but not proved; (iii) it remains open whether quantum walk algorithms or hidden-subgroup techniques [28] can exploit the algebraic structure of the cubic S-boxes in a non-associative setting. A formal quantum security reduction would be a significant contribution.

6.8 Side-Channel and Fault-Injection Attacks

The current implementation uses constant-time table lookups, but a hardware deployment would require additional protections against cache-timing attacks, power analysis, and fault injection. Masking and shuffling countermeasures — well-studied for AES [29] — should be adapted to the quasigroup round function.

6.9 Chosen-Ciphertext and Adaptive Attacks

The attack simulation (§ 4) considers a passive malicious evaluator. An adaptive attacker who can inject chosen ciphertexts into the evaluation pipeline (e.g., by modifying encrypted ML feature vectors) could potentially exploit the deterministic nature of the NAND table to mount a chosen-ciphertext attack (CCA). Designing CCA-secure variants — for instance by incorporating ciphertext authentication tags — is an important open problem.

The underlying algebraic framework — non-associative quasigroups producing Latin squares — remains a promising and under-explored foundation for homomorphic encryption. We encourage the cryptographic community to investigate these directions and welcome scrutiny, criticism, and collaboration. All source code is available in the *EquationalSecurityProject* repository.

References

1. F. Bourse, R. Del Pino, M. Minelli, H. Wee, "FHE circuit privacy almost for free," *CRYPTO 2016*, LNCS 9815, pp. 62–89.

2. K. Kluczniak, "Circuit privacy for FHEW/TFHE-style fully homomorphic encryption in practice," *IACR ePrint 2022/1459*.
3. D. Boneh, A. Sahai, B. Waters, "Functional encryption: Definitions and challenges," *TCC 2011*, LNCS 6597, pp. 253–273.
4. A. Bishop, A. Jain, L. Kowalczyk, "Function-hiding inner product encryption," *ASIACRYPT 2015*, LNCS 9452, pp. 470–491.
5. I. Chillotti, N. Gama, M. Georgieva, M. Izabachene, "TFHE: Fast fully homomorphic encryption over the torus," *J. Cryptology*, 33(1), 2020.
6. L. Ducas, D. Micciancio, "FHEW: Bootstrapping homomorphic encryption in less than a second," *EUROCRYPT 2015*, LNCS 9056, pp. 617–640.
7. R. Rivest, L. Adleman, M. Dertouzos, "On data banks and privacy homomorphisms," *Foundations of Secure Computation*, 1978.
8. C. Gentry, "Fully homomorphic encryption using ideal lattices," *STOC 2009*, pp. 169–178.
9. Z. Brakerski, C. Gentry, V. Vaikuntanathan, "(Leveled) Fully homomorphic encryption without bootstrapping," *ITCS 2012*.
10. J. Fan, F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR ePrint 2012/144*.
11. Microsoft SEAL (release 4.1), `https://github.com/microsoft/SEAL`, 2023.
12. C. Gentry, A. Sahai, B. Waters, "Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based," *CRYPTO 2013*.
13. J. H. Cheon, A. Kim, M. Kim, Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," *ASIACRYPT 2017*.
14. J. Denes, A. D. Keedwell, *Latin Squares and Their Applications*, Academic Press, 1974.
15. S. Markovski, D. Gligoroski, S. Andova, "Using quasigroups for one-one secure encoding," *Conf. Informatics and Info. Technology*, 1999.
16. V. A. Shcherbacov, "Quasigroups in cryptology," *Computer Science Journal of Moldova*, 17(2), 2009.
17. S. Kim, K. Lewi, A. Mandal, H. Montgomery, A. Roy, D. J. Wu, "Function-hiding inner product encryption is practical," *SCN 2018*, LNCS 11035, pp. 544–562.
18. S. Gorbunov, V. Vaikuntanathan, H. Wee, "Predicate encryption for circuits from LWE," *CRYPTO 2015*, LNCS 9216, pp. 503–523.
19. F. Bourse, M. Minelli, M. Minihold, P. Paillier, "Fast homomorphic evaluation of deep discretized neural networks," *CRYPTO 2018*, LNCS 10993, pp. 483–512.
20. R. Guimarães, D. F. Aranha, E. Borin, "Revisiting the functional bootstrap in TFHE," *IACR Trans. Cryptographic Hardware and Embedded Systems (TCHES)*, 2021(2), pp. 229–253.
21. A. López-Alt, E. Tromer, V. Vaikuntanathan, "On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption," *STOC 2012*, pp. 1219–1234.
22. D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, A. Sahai, "Threshold cryptosystems from threshold fully homomorphic encryption," *CRYPTO 2018*, LNCS 10991.
23. J. Patarin, "The 'Coefficients H' technique," *SAC 2008*, LNCS 5381, pp. 328–345.
24. N. Mouha, Q. Wang, D. Gu, B. Preneel, "Differential and linear cryptanalysis using mixed-integer linear programming," *Inscrypt 2011*, LNCS 7537.
25. S. Sun, L. Hu, P. Wang, K. Qiao, X. Ma, L. Song, "Automatic security evaluation and (related-key) differential characteristic search," *ASIACRYPT 2014*, LNCS 8873.

26. H. Krawczyk, P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," *RFC 5869*, 2010.
27. H. Chen, W. Dai, M. Kim, Y. Song, "Efficient homomorphic conversion between (any) FHE schemes," *IACR ePrint 2020/015*.
28. M. Ettinger, P. Hoyer, E. Knill, "The quantum query complexity of the hidden subgroup problem is polynomial," *Inform. Process. Lett.*, 91(1), 2004.
29. S. Mangard, E. Oswald, T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*, Springer, 2007.